SE446 Milestone 2 - Task 9 Local Execution

Author: Mishari Al Mogren

This notebook is for Task 9 evidence. It runs locally in Google Colab using Spark `local[*]` and generated 10,000-row sample data.

```
# ============================================
# Task 9 Setup: Install PySpark
# Author: Mishari Al Mogren
# ============================================

!pip -q install pyspark
```

```
# ============================================
# Task 9 Setup: Start Spark in local mode
# Author: Mishari Al Mogren
# ============================================

from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("SE446_M2_Task9_Local") \
    .master("local[*]") \
    .config("spark.sql.shuffle.partitions", "4") \
    .getOrCreate()

spark.sparkContext.setLogLevel("WARN")

print(f"Spark version: {spark.version}")
print(f"Master: {spark.sparkContext.master}")
```

```
Spark version: 4.0.2
Master: local[*]
```

```
# ============================================
# Task 9 Data: Generate 10,000 rows using Lab 9 pattern
# Author: Mishari Al Mogren
# ============================================

from pyspark.sql import Row
from pyspark.sql.functions import col, monotonically_increasing_id,
import random
```

```
random.seed(42)

crime_profiles = {
    "NARCOTICS":        0.85,
    "PROSTITUTION":     0.80,
    "WEAPONS VIOLATION":0.60,
    "BATTERY":          0.30,
    "ASSAULT":          0.25,
    "ROBBERY":          0.15,
    "THEFT":            0.10,
    "BURGLARY":         0.08,
    "MOTOR VEHICLE THEFT": 0.06,
    "CRIMINAL DAMAGE":  0.05,
}

districts = list(range(1, 26))

def generate_row():
    crime_type = random.choice(list(crime_profiles.keys()))
    base_rate = crime_profiles[crime_type]
    district = random.choice(districts)
    hour_value = random.randint(0, 23)
    domestic = random.random() < 0.15

    arrest_prob = base_rate
    if domestic:
        arrest_prob += 0.20
    if 2 <= hour_value <= 5:
        arrest_prob -= 0.10
    arrest_prob = max(0.01, min(0.99, arrest_prob))
    arrest = random.random() < arrest_prob

    return Row(
        District=district,
        PrimaryType=crime_type,
        Hour=hour_value,
        Domestic=domestic,
        Arrest=arrest
    )

# Generate 10,000 rows exactly.
rows = [generate_row() for _ in range(10000)]
lab_df = spark.createDataFrame(rows)

raw_df = lab_df.withColumn("ID", monotonically_increasing_id() + 900
        "Location Description",
        when(col("District") <= 5, "STREET")
        .when(col("District") <= 10, "RESIDENCE")
        .when(col("District") <= 15, "APARTMENT")
        .when(col("District") <= 20, "SIDEWALK")
        .otherwise("OTHER")
    ).withColumn("Year", when(col("Hour") < 6, 2021).when(col("
```

```
    )     .withColumn("Year", when(col("Hour") < 6, 2021).when(col("

print(f"Generated rows: {raw_df.count()}")
print("Lab 9 base dataframe:")
lab_df.show(5, truncate=False)
print("Milestone-compatible dataframe:")
raw_df.select("ID", "Primary Type", "Location Description", "Arrest"
```

```
Generated rows: 10000
Lab 9 base dataframe:
+--------+------------------+----+--------+------+
|District|PrimaryType       |Hour|Domestic|Arrest|
+--------+------------------+----+--------+------+
|1       |PROSTITUTION      |23  |false   |true  |
|22      |PROSTITUTION      |23  |false   |true  |
|2       |THEFT             |0   |true    |true  |
|1       |CRIMINAL DAMAGE   |17  |false   |false |
|14      |MOTOR VEHICLE THEFT|7   |false   |false |
+--------+------------------+----+--------+------+
only showing top 5 rows
Milestone-compatible dataframe:
+--------+------------------+--------------------+------+--------+-
|ID      |Primary Type      |Location Description|Arrest|Domestic|D
+--------+------------------+--------------------+------+--------+-
|90000000|PROSTITUTION      |STREET              |true  |false   |1
|90000001|PROSTITUTION      |OTHER               |true  |false   |2
|90000002|THEFT             |STREET              |true  |true    |2
|90000003|CRIMINAL DAMAGE   |STREET              |false |false   |1
|90000004|MOTOR VEHICLE THEFT|APARTMENT           |false |false   |1
+--------+------------------+--------------------+------+--------+-
only showing top 5 rows
```

```
# ============================================
# Task 9 Data: Clean and prepare dataframe
# Author: Mishari Al Mogren
# ============================================

from pyspark.sql.functions import col, count, avg, lower, when

df = raw_df.withColumn(
    "District",
    col("District").cast("double")
).withColumn(
    "Domestic_str",
    lower(col("Domestic").cast("string"))
).withColumn(
    "label",
    when(lower(col("Arrest").cast("string")) == "true", 1).otherwis
).select(
    "ID", "District", "Primary Type", "Location Description",
    "Hour", "Domestic_str", "Year", "label"
).dropna()

df.cache()

print(f"Rows after cleaning: {df.count()}")
df.groupBy("label").count().orderBy("label").show()
```

```
Rows after cleaning: 10000
+-----+-----+
|label|count|
+-----+-----+
|    0| 6595|
|    1| 3405|
+-----+-----+
```

Phase A - Spark DataFrame Analytics

```
# ============================================
# Task 1: Crime Type Distribution
# Author: Abdulaziz Albaz
# ============================================

print("Task 1 - Top 10 crime types")
df.groupBy("Primary Type") \
    .count() \
    .orderBy(col("count").desc()) \
    .show(10, truncate=False)
```

```
Task 1 - Top 10 crime types
+------------------+-----+
|Primary Type      |count|
+------------------+-----+
|BURGLARY          |1040 |
|ROBBERY           |1020 |
|THEFT             |1016 |
|WEAPONS VIOLATION |1008 |
|PROSTITUTION      |1005 |
|CRIMINAL DAMAGE   |1001 |
|BATTERY           |994  |
|ASSAULT           |990  |
|NARCOTICS         |964  |
|MOTOR VEHICLE THEFT|962  |
+------------------+-----+
```

```
# ============================================
# Task 2: Location Hotspots using Spark SQL
# Author: Abdulaziz Alnemer
# ============================================

print("Task 2 - Top 10 locations using Spark SQL")
df.createOrReplaceTempView("crimes")

spark.sql("""
    SELECT `Location Description`, COUNT(*) AS total
    FROM crimes
    GROUP BY `Location Description`
    ORDER BY total DESC
    LIMIT 10
""").show(10, truncate=False)
```

```
Task 2 - Top 10 locations using Spark SQL
+--------------------+-----+
|Location Description|total|
+--------------------+-----+
|OTHER               |2055 |
|APARTMENT           |2009 |
|SIDEWALK            |1995 |
|RESIDENCE           |1980 |
|STREET              |1961 |
+--------------------+-----+
```

```
# ============================================
# Task 3: Crime Trend Over Years
# Author: Ibrahim Alhagbani
# ============================================

print("Task 3 - Crime count by year")
yearly = df.groupBy("Year").count().orderBy("Year")
yearly.show(50, truncate=False)

print("Interpretation: in this generated local sample all rows are
```

```
Task 3 - Crime count by year
+----+-----+
|Year|count|
+----+-----+
|2021|2565 |
|2022|2408 |
|2023|2515 |
|2024|2512 |
+----+-----+

Interpretation: in this generated local sample all rows are from 2024
```

```
# ============================================
# Task 4: Arrest Rate Analysis
# Author: Nawaf Alshuaibi
# ============================================

total = df.count()
arrests = df.filter(col("label") == 1).count()
overall_rate = arrests / total

print("Task 4 - Arrest rate analysis")
print(f"Total rows: {total}")
print(f"Arrest rows: {arrests}")
print(f"Overall arrest rate: {overall_rate:.4f}")

print("Arrest rate by crime type")
df.groupBy("Primary Type") \
    .agg(count("*").alias("total"), avg("label").alias("arrest_rate
    .orderBy(col("arrest_rate").desc()) \
    .show(10, truncate=False)

print("Interpretation: the generated data gives higher arrest rates
```

```
Task 4 - Arrest rate analysis
Total rows: 10000
Arrest rows: 3405
Overall arrest rate: 0.3405
Arrest rate by crime type
+------------------+-----+------------------+
|Primary Type      |total|arrest_rate       |
+------------------+-----+------------------+
|NARCOTICS         |964  |0.8692946058091287|
|PROSTITUTION      |1005 |0.8019900497512438|
|WEAPONS VIOLATION |1008 |0.6121031746031746|
|BATTERY           |994  |0.3118712273641851|
|ASSAULT           |990  |0.2696969696969697|
|ROBBERY           |1020 |0.16862745098039217|
|THEFT             |1016 |0.11318897637795275|
|BURGLARY          |1040 |0.10480769230769231|
|MOTOR VEHICLE THEFT|962 |0.08835758835758836|
|CRIMINAL DAMAGE   |1001 |0.08591408591408592|
+------------------+-----+------------------+

Interpretation: the generated data gives higher arrest rates to cate
```

Phase B - Spark MLlib

```
# ============================================
# Task 5: Feature Engineering Pipeline
# Author: Ibrahim AlHagbani
```

```
# ==========================================

from pyspark.ml import Pipeline
from pyspark.ml.feature import StringIndexer, VectorAssembler

feature_cols = ["District", "crime_index", "Hour", "domestic_index"

crime_indexer = StringIndexer(
    inputCol="Primary Type",
    outputCol="crime_index",
    handleInvalid="skip"
)

domestic_indexer = StringIndexer(
    inputCol="Domestic_str",
    outputCol="domestic_index",
    handleInvalid="skip"
)

assembler = VectorAssembler(
    inputCols=feature_cols,
    outputCol="features"
)

feature_pipeline = Pipeline(stages=[crime_indexer, domestic_indexer
feature_model = feature_pipeline.fit(df)
feature_df = feature_model.transform(df)

print("Task 5 - Feature vector sample")
feature_df.select(
    "Primary Type", "District", "Hour", "Domestic_str",
    "crime_index", "domestic_index", "features", "label"
).show(5, truncate=False)

print("Feature vector order: [District, crime_index, Hour, domestic

train_df, test_df = df.randomSplit([0.8, 0.2], seed=42)
train_df.cache()
test_df.cache()

print(f"Training rows: {train_df.count()}")
print(f"Testing rows: {test_df.count()}")
```

```
Task 5 - Feature vector sample
+------------------+--------+----+------------+-----------+--------
|Primary Type      |District|Hour|Domestic_str|crime_index|domestic
+------------------+--------+----+------------+-----------+--------
|PROSTITUTION      |1.0     |23  |false       |4.0        |0.0
|PROSTITUTION      |22.0    |23  |false       |4.0        |0.0
|THEFT             |2.0     |0   |true        |2.0        |1.0
|CRIMINAL DAMAGE   |1.0     |17  |false       |5.0        |0.0
```

```
|MOTOR VEHICLE THEFT|14.0    |7   |false       |9.0         |0.0
+-------------------+--------+----+------------+------------+--------
only showing top 5 rows
Feature vector order: [District, crime_index, Hour, domestic_index]
Training rows: 8053
Testing rows: 1947
```

```
# ============================================
# Task 6: Train and Evaluate Three Models
# Author: Abdulaziz Albaz
# ============================================

import time
from pyspark.ml.classification import LogisticRegression, RandomFor
from pyspark.ml.evaluation import BinaryClassificationEvaluator, Mu

binary_eval = BinaryClassificationEvaluator(
    labelCol="label",
    rawPredictionCol="rawPrediction",
    metricName="areaUnderROC"
)

multi_eval = MulticlassClassificationEvaluator(
    labelCol="label",
    predictionCol="prediction"
)

def get_confusion(predictions):
    rows = predictions.groupBy("label", "prediction").count().colle
    values = {(int(r["label"]), int(r["prediction"])): r["count"] f
    return (
        values.get((0, 0), 0),
        values.get((0, 1), 0),
        values.get((1, 0), 0),
        values.get((1, 1), 0),
    )

def train_and_score(name, classifier):
    pipeline = Pipeline(stages=[crime_indexer, domestic_indexer, as
    start = time.time()
    model = pipeline.fit(train_df)
    train_time = time.time() - start
    predictions = model.transform(test_df)
    tn, fp, fn, tp = get_confusion(predictions)

    return model, {
        "Model": name,
        "AUC": binary_eval.evaluate(predictions),
        "Accuracy": multi_eval.evaluate(predictions, {multi_eval.me
        "F1": multi_eval.evaluate(predictions, {multi_eval.metricNa
```

```
        "Precision": multi_eval.evaluate(predictions, {multi_eval.m
        "Recall": multi_eval.evaluate(predictions, {multi_eval.metr
        "Training Time": train_time,
        "TN": tn,
        "FP": fp,
        "FN": fn,
        "TP": tp,
    }

models = [
    (
        "Logistic Regression",
        LogisticRegression(featuresCol="features", labelCol="label"
    ),
    (
        "Random Forest",
        RandomForestClassifier(featuresCol="features", labelCol="la
    ),
    (
        "GBT",
        GBTClassifier(featuresCol="features", labelCol="label", max
    ),
]

trained = {}
results = []

for name, classifier in models:
    print(f"Training {name}...")
    model, row = train_and_score(name, classifier)
    trained[name] = model
    results.append(row)

print("Task 6 - Model comparison")
print(f"{'Model':<22} {'AUC':>7} {'Acc':>7} {'F1':>7} {'Prec':>7} {
print("-" * 92)

for row in results:
    print(
        f"{row['Model']:<22} {row['AUC']:>7.3f} {row['Accuracy']:>7
        f"{row['F1']:>7.3f} {row['Precision']:>7.3f} {row['Recall']
        f"{row['Training Time']:>8.2f} {row['TN']:>4} {row['FP']:>4
        f"{row['FN']:>4} {row['TP']:>4}"
    )

for row in results:
    for metric in ["AUC", "Accuracy", "F1", "Precision", "Recall"]:
        assert 0.0 <= row[metric] <= 1.0

print("Validation: all ML metrics are between 0 and 1.")
print("Interpretation: compare models mainly by AUC and F1 because
```

```
Training Logistic Regression...
Training Random Forest...
Training GBT...
Task 6 - Model comparison
Model                      AUC     Acc      F1    Prec  Recall     T
-------------------------------------------------------------------
Logistic Regression      0.597   0.662   0.570   0.633   0.662     5
Random Forest            0.865   0.818   0.814   0.815   0.818     5
GBT                      0.863   0.814   0.810   0.810   0.814    32
Validation: all ML metrics are between 0 and 1.
Interpretation: compare models mainly by AUC and F1 because accuracy
```

```
# ============================================
# Task 7: Feature Importance
# Author: Nawaf Alshuaibi
# ============================================

rf_model = trained["Random Forest"].stages[-1]
importances = list(rf_model.featureImportances)
pairs = sorted(zip(feature_cols, importances), key=lambda item: ite

print("Task 7 - Random Forest feature importance ranking")
for name, value in pairs:
    print(f"{name:<16} {value:.4f} " + "#" * int(value * 40))

print(f"Most important feature: {pairs[0][0]}")
print("Interpretation: if crime_index is highest, the model is lear
```

```
Task 7 - Random Forest feature importance ranking
crime_index      0.9417 ######################################
domestic_index   0.0397 #
Hour             0.0122
District         0.0064
Most important feature: crime_index
Interpretation: if crime_index is highest, the model is learning tha
```

```
# ============================================
# Task 9 Final Evidence Summary
# Author: Mishari Al Mogren
# ============================================

print("TASK 9 EVIDENCE SUMMARY")
print(f"Master: {spark.sparkContext.master}")
print(f"Generated rows: {raw_df.count()}")
print(f"Rows after cleaning: {df.count()}")
print("Task 9 complete: notebook ran locally with generated 10,000-
```

```
TASK 9 EVIDENCE SUMMARY
Master: local[*]
Generated rows: 10000
Rows after cleaning: 10000
Task 9 complete: notebook ran locally with generated 10,000-row data
```